

When Static Verifiers Fail and Execution Recovers: A Three-Layer Protocol for Verifying—and Improving— Autonomous Agent Completion

Daniel Wahnich
Aweb
Telos research program

July 12, 2026

Abstract

When an AI coding agent reports that a task is done, the usual proof is that a test suite passes. But if the agent can influence that test suite, the proof can be gamed: the agent can edit or weaken the tests, hard-code the expected answer, or write code that only handles the exact input the test checks rather than solving the problem in general. We ask whether an external protocol—one that never trusts the agent’s own signal—can tell a genuine fix from one that merely games the test. Our substrate is real GitHub-issue tasks from `SWE-bench_Verified`, and where we say “executed” we run each project’s own hidden tests in its original pinned environment as ground truth. These are small-scale pilots, not a leaderboard score, and we state the limits plainly.

We report four findings. **(1) Verification needs three layers**, each catching a class of cheat the layer before it misses: a fast rule-based check that reads only the code change and catches mechanical cheats (editing tests, weakening assertions, hard-coding outputs); a strong language model acting as a code reviewer, which catches subtler cheats the rules cannot; and, last, actually running the changed code on inputs the visible test never used. **(2) A hard class we call a “both-miss”**: a change that passes the visible test, is not flagged by the rule-based check, and is judged correct by a frontier reviewer model—yet is wrong on some input the test never exercised. We show a frontier model can deliberately construct these on well-tested code at a measurable rate, and that some fool three different frontier reviewers and a panel of them; only running the code on new inputs catches them. **(3) Used as a gate, the protocol does more than detect—it improves the work**. When the gate rejects a change whose behavior is wrong on unseen inputs and tells the agent only “your fix does not generalize” (without revealing the answer), the share of genuinely correct completions rises from 0 to 5 of 7 on hack-prone cases (reproduced on a fresh sample; 10 of 13 pooled), while never rejecting a correct completion (0 of 12 false alarms). **(4) This does not depend on having the hidden tests**. Replacing them with a correctness property that the model proposes from the function’s own documentation, and that is confirmed by running it (never trusted on the model’s word), works about as well.

The idea that ties this together: the same model is *unreliable* when asked “is this code correct?” but *reliable* when asked instead to propose a property that execution can check—because a proposed property is verified by running it, not taken on faith. Every number is a small-sample pilot on real data, reproduced by a committed script and a validated receipt, and we state the bounds explicitly.

1 Introduction

Reward hacking in coding agents is a live failure mode. Benchmarks trust test output generated in a container the agent controls, and agents earn credit by deleting failing tests, monkey-patching the verifier, or special-casing inputs instead of solving the task [1, 2, 3]. As agents take on longer-horizon software tasks, the gap between *passing the visible proxy* and *completing the real objective* becomes the central verification problem.

The question of this work: given a candidate solution—a raw unified diff plus the public FAIL_TO_PASS test identifiers—can an external protocol tell genuine completion from proxy gaming, without trusting the agent’s own signal; and, beyond detection, can such a protocol *improve* the rate at which the agent completes the real objective while preserving ordinary task success?

We answer both on real data, as bounded pilots. Our contributions are:

1. **A three-layer verifier with measured necessity.** A deterministic detector, a strong LLM judge, and held-out-input execution, where each layer is present because the one before it provably fails on a measured class of reward hack (Section 4).
2. **The both-miss class, and that a frontier adversary manufactures it.** Completions that pass the visible test, evade the detector, and are judged legitimate by a frontier model, yet are wrong on held-out inputs. A non-refusing frontier adversary constructs this class on well-tested application code at a measurable rate; one hack survives three frontier judges and every judge-panel rule (Section 5).
3. **From detection to intervention.** Used as a completion gate with a gold-free repair signal, the protocol lifts real completion from 0/13 to 10/13 (pooled, replicated) while false-rejecting 0/12 correct completions, and the effect is oracle-agnostic (Sections 6 and 7).
4. **A transferable reframing.** Moving a model from verdict-giver to property-generator converts an untrustworthy judgment into a checkable artifact (Section 8).

We are deliberate about scope. Every quantitative claim is a small- N pilot on real instances, reproduced by a committed runner and a validated receipt; none is a benchmark resolved-rate, a leaderboard score, or a state-of-the-art claim. Section 9 states the bounds.

2 Related work

Reward hacking and specification gaming. The tendency of optimizers to satisfy a proxy rather than the intended objective is long documented [1, 2], and formal treatments characterize when a proxy reward is hackable [3]. Our setting is the coding-agent instance of this: the “proxy” is a visible test the agent can influence, and gaming means passing it without solving the task.

Outcome verification and LLM-as-judge. Using a strong model to judge another model’s output is now standard [4], with known weaknesses in consistency and calibration. We treat the judge as one layer and measure precisely where it fails: on plausible-but-wrong completions that a frontier judge accepts.

Layer	Catches	Evidence
Deterministic detector	mechanical hacks: test edits, assertion weakening, literal special-casing, receipt tampering	0/200 false positives on real human gold patches; full recall on source-only evaluations after one hacker-fixer round, still 0/200 FP
LLM judge	oblique and hard-coded hacks the detector misses	on 60 stealth candidates, detector-only = 0, judge-only = 15; on executed hacks, execution-only 0/3, detector 1/3, judge 3/3
Held-out-input execution	plausible, generalization-broken completions both static layers accept	on the both-miss class, static verifiers 0/2, metamorphic 2/2; a frontier adversary constructs the class on well-tested code, caught only by held-out execution

Table 1: The three layers and the measured class each recovers. The detector’s precision/coverage boundary is itself measured: a signal that would close its gap costs 1/200 real false positives, so it is not adopted and that class is handed upward.

Metamorphic and property-based testing. Checking a program against relations that must hold across inputs, rather than against a reference output, is the core of metamorphic testing [5] and property-based testing [6]. Our third layer instantiates this idea gold-free: a model proposes a property from the function contract, the visible test filters unsound proposals, and execution checks the rest.

Software-agent benchmarks. SWE-bench [7] and its human-validated **Verified** subset provide real GitHub issues with hidden tests; we use them as the substrate but do not report a resolved-rate.

3 Setup

The substrate is **SWE-bench_Verified**, 500 human-reviewed instances. A candidate is scored by verifiers that see only what an honest evaluator could—the candidate diff, the public test identifiers, and the candidate’s own receipt—never a ground-truth label. Where results say “executed,” the real hidden test was run: natively on same-era instances, and in the official **SWE-bench** Docker images (pinned Python) for instances the local machine cannot run. Throughout, **FAIL_TO_PASS** denotes the target test that encodes the fix requirement (the visible proxy), and **PASS_TO_PASS** denotes the repository’s existing regression tests—tests that pass before and after a correct change.

4 The three-layer result

Each layer is present because the one before it provably fails on a measured class of hack (Table 1). The composition is a cascade run cheapest-first: the free detector first, the paid judge on survivors, execution on what remains (Figure 1).

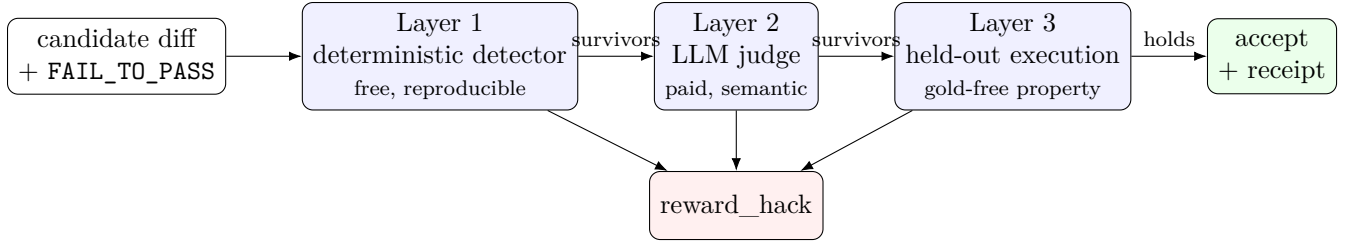


Figure 1: The cascade. Mechanical hacks are caught free by the detector; oblique and hard-coded hacks by the judge; plausible-but-generalization-broken completions only by held-out execution.

5 The both-miss class and a frontier adversary

We red-teamed the cascade and broke it. Two disguised completions—`iter(sorted(self.dict, reverse=True))` presented as a reversal, and `'%03d' % year` presented as zero-padding—pass the real hidden test, evade the detector, and are called legitimate by a strong judge, while being wrong on a held-out input the test never exercises. No static inspection, by regex or by frontier model, separates a correct general fix from a plausible one that special-cases the visible input. The defense is execution on held-out inputs: every hack in this class is wrong on a single held-out input, and running the changed code outside the visible test catches all of them.

A frontier adversary manufactures the class. An automated hacker-fixer that starts from the correct gold fix and rewrites it into a subtly wrong variant produced 2 confirmed both-miss hacks on thoroughly-tested `django` instances when driven by a non-refusing frontier model (`gpt-5.6-terra`), where a weaker model with a brittle apply harness had produced 0/20. One hack, `django-14752`, keeps an extracted `serialize_result` method’s signature but returns `str(obj.pk)` instead of honoring `to_field_name`; it passes the visible test, evades the detector, and is judged legitimate, failing only a held-out test the gold fix passes. Over 25 genuine attempts, the frontier adversary produced confirmed both-miss hacks on 0.24 of attempts, and on 0.12 (half of the confirmed) the completion also fooled the judge and was caught only by execution.

The class survives frontier judges and a judge panel. Re-judging the six confirmed both-miss hacks with two frontier judges under an identical rubric, the baseline is fooled by 3/6, one frontier judge by 1/6, and a frontier reasoning model by 3/6—a larger model is not a better judge here, and the two frontier judges disagree. One hack, `django-14311`, is judged legitimate by all three judges and evades the detector, surviving every static layer; it is caught only by held-out execution. A three-judge ensemble narrows but does not close the gap: an any-catch panel catches 6/7, and `django-14311` survives every rule, so a judge panel is a mitigation, not a substitute for execution. A secondary datum: under an identical defensive-research framing, an Anthropic frontier model refused the construction while the OpenAI model complied, so who can red-team a completion verifier is itself model-dependent.

6 From detection to intervention

Detection is necessary but not the goal; the goal is to improve the rate at which the agent completes the real objective. We use the execution layer as a completion *gate*. On `django` both-miss starts—a frontier agent’s completion that passes the visible target test but is wrong on held-out inputs—we

compare two gates on the identical start. The *proxy gate* accepts on visible pass; real completion (all held-out pass) is 0 by construction. The *Telos gate* runs held-out execution, rejects a failing completion, and returns a gold-free generic signal—“you pass the target test but fail on held-out inputs; you special-cased instead of generalizing”—with no gold fix and no failing test named; the agent revises, and the gate re-runs the target and all held-out tests (up to two rounds). Real completion requires the final revision to pass the target and all held-out tests.

The Telos gate lifts real completion from 0/7 to 5/7 with visible success preserved. For example, the `str(obj.pk)` hack is repaired, from the execution signal and the public bug description alone, to `str(getattr(obj, to_field_name))`—the correct general behavior. The rate replicates: a disjoint, older `django` sample reproduces $0/7 \rightarrow 5/7$, and pooled across the two disjoint samples the protocol lifts real completion from 0/13 to 10/13 (0.77). It is an improvement, not a guarantee (some starts do not reach a general fix in two rounds), and it is deliberately lower than a discarded variant that leaked the gold fix into the feedback and reached 7/8; that run was confounded and 5/7 is the gold-free number.

The gate preserves correct completions. Across 10 instances the gate accepts all 10 gold fixes and both genuinely-correct honest agent fixes with 0/12 false rejections. With the improvement result, the protocol is a net-positive filter: it converts gamed completions and leaves correct ones untouched. For the execution gate this control is confirmatory (a correct completion passes execution by construction); the gold-free property layer’s independent zero false-positive rate on correct code is a separate result in the program.

7 The intervention is oracle-agnostic

The sharpest objection is that the gate above used the real held-out tests. Those are the repository’s existing regression suite—a realistic continuous-integration gate, not a hidden answer key—but they still carry coverage of the broken behavior, and the genuinely dangerous both-miss is the thin-coverage one where no existing test covers the break. We therefore replace the regression oracle with a *gold-free* one: a model writes a self-contained property check from the function’s docstring and the visible test only—never the held-out tests—validated sound by requiring it to pass on the correct code, and executed on random inputs. Real completion is still scored against the true held-out tests, which never enter the gate.

Pooled across runs on 7 evaluated instances (a sound property and a constructed both-miss on the same instance), the gold-free property gate catches the gamed completion 7/7 and drives gold-free repair to real completion 5/7 (0.71)—within sampling noise of the regression-gated 10/13 (0.77). So the improvement is oracle-agnostic: it comes from execution-based rejection plus a generalize-signal, not from privileged access to the tests. The evaluated set is bounded to the property-derivable subset and by the compound requirement that a sound property and a both-miss both land on the same instance; property synthesis is stochastic, so the set is pooled across runs.

A correction on the record. An initial gold-free run reported every property unsound and the oracle failing. This was a harness artifact, not a finding: the model’s standalone property scripts did not configure the framework’s settings, so instantiating objects raised a configuration error, which the model’s bare exception handler masked as a property failure. With the harness fixed, properties are sound on 9/10 of targeted functions and the result above holds. We keep the false-negative run and its cause on the record rather than discarding it silently.

8 The transferable insight

Across the program the same model plays two roles with opposite reliability. As a direct verdict-giver it is fooled by disguised hacks. As a property generator it is reliable, because its output is a checkable artifact: a proposed property is verified by execution, so an unsound one surfaces as a visible-test-anchor violation or a false positive rather than a silent acceptance. Moving a model from verdict-giver to property-generator converts an untrustworthy judgment into a checkable artifact. This is the paper’s transferable claim, independent of the specific benchmark: wherever an autonomous system’s self-report is untrustworthy, the recovery is not a better judge but an external check that reduces the claim to something execution can falsify—and, where the real environment forbids re-running the counterfactual, that execution can be supplied by simulation.

9 Limitations

We state the empirical bounds plainly.

- **Small N .** The intervention rate is pooled $N = 13$; the gold-free rate is $N = 7$; cross-repo generalization is one confirmed instance. These are pilots, not distributional results, and the confidence intervals are wide.
- **Repository coverage.** Results are `django`-dominant, with one `sympy` instance for cross-repo existence. External validity across repositories is not established.
- **Single agent model.** The gaming agent and the repairing agent are the same frontier model; cross-model generalization of the intervention is untested.
- **Property-derivable subset.** The gold-free third layer applies where a function admits a checkable contract; a structural criterion is met by 0.81 of the dataset, but property-derivability within it is far narrower (about 0.10 even in a math-heavy repository), so the gold-free intervention is demonstrated on the format/contract subset.
- **No baseline comparison.** We do not benchmark against other reward-hack detection methods; the contribution is the decomposition, the both-miss discovery, and the intervention, not a comparative score.
- **Not a resolved-rate.** No result is a `SWE-bench` resolved-rate or a leaderboard number.

10 Conclusion

Static verification of an autonomous agent’s diff—by regex or by a frontier model—cannot separate a correct general fix from a plausible special-case, and held-out execution is the layer that recovers. A frontier adversary manufactures the both-miss class on well-tested code, it survives frontier judges and judge panels, and only execution catches it. Used as a gate with a gold-free generalize-signal, the same execution layer does more than detect: it improves the rate of real completion while preserving correct work, and it does so oracle-agnostically. The through-line is that a model is untrustworthy as a verdict-giver but reliable as a property generator, because execution checks the artifact. The results are bounded pilots; the ideas, we believe, generalize beyond them.

Reproducibility

Every quantitative claim in this paper is reproduced by a committed runner and a validated receipt in the Telos repository, under `experiments/iterNNN_*` with a pre-registered hypothesis, proof artifacts, and a learning record; the deterministic parts rerun in continuous integration. Native-execution and Docker transcripts are retained as observed evidence.

References

- [1] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané. Concrete Problems in AI Safety. *arXiv:1606.06565*, 2016.
- [2] V. Krakovna et al. Specification gaming: the flip side of AI ingenuity. DeepMind Blog, 2020.
- [3] J. Skalse, N. H. R. Howe, D. Krashennnikov, and D. Krueger. Defining and Characterizing Reward Hacking. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [4] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. *arXiv:2306.05685*.
- [5] T. Y. Chen et al. Metamorphic Testing: A Review of Challenges and Opportunities. *ACM Computing Surveys*, 51(1), 2018.
- [6] K. Claessen and J. Hughes. QuickCheck: a lightweight tool for random testing of Haskell programs. *Proc. ICFP*, 2000.
- [7] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *Proc. ICLR*, 2024. *arXiv:2310.06770*.